Ders 15 Genişletilmiş Notu: Öncelikli Kuyruklar ve Huffman Kodlama

Ders 15 Genişletilmiş Notu: Öncelikli Kuyruklar ve Huffman Kodlama

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Heap Sort'un Tekrarı
- 2. Sıralama Algoritmalarının Karşılaştırılması
- 3. Öncelikli Kuyruk ve Gerçekleştirme Seçenekleri
- 4. Heap ile Öncelikli Kuyruk İşlemleri
- 5. Huffman Kodlama
- 6. Sınava Hazırlık: Heap ve BST Soruları
- Örnek Sorular
- Dersin Toparlaması

1. Heap Sort'un Tekrarı

Hoca derse, geçen hafta görülen heap sort'u hatırlatarak başladı. Bugünkü konu da heap ağacı (heap tree) üzerine kurulacağı için önce sıralamanın nasıl yapıldığını bir örnek üzerinden yeniden izledi.

1.1 Başlangıç Ağacı

Ekrandaki ağaç bir maksimum heap ağacıdır (max heap tree). Max heap'te her ebeveyn (parent) çocuklarından (child) büyüktür. Örnekte 10, kendi çocukları olan 6 ve 3'ten büyüktür. 22 de çocukları 13 ve 5'ten büyüktür. Kökte 45 vardır.

Ağaç bir dizide şöyle saklanır: kök 0. adrese, onun çocukları 1. ve 2. adreslere, onların çocukları 3, 4, 5, 6. adreslere yerleşir.

Adres	0	1	2	3	4	5	6
Değer	45	10	22	6	3	13	5

1.2 Sıralama Adımları

Max heap'te kökteki eleman en büyüktür, çünkü her ebeveyn çocuklarından büyükse kökteki eleman herkesten büyük demektir. O hâlde kökü dizinin sonuna taşırsak, onu sıralı dizide olması gereken yere koymuş oluruz.

İlk adımda 45 ile son eleman 5 yer değiştirir. 45 artık doğru yerindedir ve bundan sonra ona bakılmaz. Ama 5'in kökte durması heap özelliğini bozar, çünkü 5 hem 10'dan hem 22'den küçüktür. Bu yüzden 5 aşağı taşınır (heapify):

- 10 ile 22'den hangisi büyükse o yukarı alınır. 22 büyüktür, 22 ile 5 yer değiştirir.
- 5'in yeni yerinde tek çocuk vardır: 13. 13 ile 5 karşılaştırılır ve yer değiştirir.

- Dizinin yeni hâli: `22, 10, 13, 6, 3, 5 | 45`.

İkinci adımda kök 22, bakılan bölgenin son elemanı 5 ile yer değiştirir. 5 yine aşağı taşınır: önce 10 ile 13'ten büyük olan bulunur (13), 13 ile 5 yer değiştirir. 5'in altında çocuk kalmadığı için işlem biter. Dizi `13, 10, 5, 6, 3 | 22, 45` olur.

Bu şekilde devam edilir. En son adımda 5 ile 3 kalır. Bunlar da yer değiştirince dizi küçükten büyüğe sıralanmış olur. Bütün adımlar (python ile doğrulandı):

i	Değişimden ve sift down'dan sonra bakılan bölge	Yerleşmiş kısım
6	22, 10, 13, 6, 3, 5	45
5	13, 10, 5, 6, 3	22, 45
4	10, 6, 5, 3	13, 22, 45
3	6, 3, 5	10, 13, 22, 45
2	5, 3	6, 10, 13, 22, 45
1	3	5, 6, 10, 13, 22, 45

Yapılan iş şudur: 0. eleman ile bakılan bölgenin son elemanı, yani i. eleman yer değiştirir. Döngü $i = n - 1$'den başlar ve her adımda bir azalır. Böylece her seferinde dizinin tamamına değil, giderek küçülen bir bölümüne bakılır. Başlangıçta beş elemanla ilgilenirken bir adım sonra dört, sonra üç elemana bakılır.

1.3 Sift Down ve Sift Up

Hoca iki heapify işlemini yeniden adlandırdı:

- **Sift down:** Aşağıya doğru heapify. Kökteki (ya da bir düğümdeki) eleman, çocuklarından büyük olana kadar aşağı taşınır. Heap sort'ta ve kökün silinmesinde kullanılır.
- **Sift up:** Yukarıya doğru heapify. Sona eklenen eleman, ebeveyninden küçük olana kadar yukarı taşınır. Ekleme (insert) işleminde kullanılır.

1.4 Heapify'ın Karmaşıklığı

Hoca sınıfa heapify işleminin, örneğin kökteki bir elemanı aşağı taşımanın karmaşıklığını sordu.

Soru: Heapify'ın karmaşıklığı $O(\log n)$ olabilir mi? **Cevap:** Evet, $O(\log n)$'dir.

Hoca nedenini önceki derste görülen bir ekleme örneğiyle açıkladı. 35 ağaca eklenip sift up yapılırken önce 35 ile ebeveyni 12 karşılaştırılır, yer değiştirir, sonra 35 ile 20 karşılaştırılır. Bu sırada ağacın öbür yarısına hiç bakılmaz. Her adımda bakılan bölgenin yarısı elenir. Sift down için de durum aynıdır.

> **Hocanın vurgusu:** Sift down da olsa sift up da olsa, heap özelliğini yeniden sağlamak için yapılan aşağı ya da yukarı taşıma işleminin karmaşıklığı her zaman $O(\log n)$'dir, çünkü her adımda bakılan bölgenin yarısı elenir.
>
> **Ek bilgi (derste anlatılmadı):** Heap, dengeli ve sola dayalı (tam) bir ikili ağaçtır. n düğümlü böyle bir ağacın yüksekliği $\lfloor \log_2 n \rfloor$'dir. Sift up ve sift down her adımda bir seviye ilerlediği için en fazla bu kadar yer değiştirme yapar. $O(\log n)$ sınırı buradan gelir.

2. Sıralama Algoritmalarının Karşılaştırılması

2.1 Best, Worst ve Average Case

Hoca, performansı değerlendirmeden önce üç durumu tanımladı:

- **Best case:** Her şey yolunda giderse olan durumdur. Sıralamada dizinin zaten sıralı verilmesidir.

- **Worst case:** En kötü durumdur. Sıralamada, küçükten büyüğe sıralamak istediğiniz dizinin büyükten küçüğe sıralı verilmesidir.
- **Average case:** Elemanların sırasının rastgele olduğu, gerçek hayatta ortalamada karşılaşılabilecek durumu temsil eder. Hoca bunun ayrıntısının algoritma analizi dersinde görüleceğini söyledi ve average case'in "ortalamasını almak" olmadığını özellikle belirtti.

  Ek bilgi (derste anlatılmadı): Average case karmaşıklığı, n boyutlu bütün girişler üzerinden (sıralamada genellikle bütün permütasyonların eşit olasılıklı olduğu varsayılarak) çalışma süresinin beklenen değeridir. Yani best ve worst case sürelerinin ortalaması değildir; girişlerin dağılımına göre hesaplanır.

2.2 Best Case: Dizi Zaten Sıralıysa

- **Selection sort:** Her adımda bakılan bölgenin en küçük elemanını bulup başa taşır. En küçüğü bulmak için her seferinde bölgenin tamamına bakmak zorundadır. Dizinin sıralı ya da sırasız olması bir şey değiştirmez. Karmaşıklık $O(n^2)$'dir.
- **Insertion sort:** Her eleman kendinden öncekilerle karşılaştırılır. Dizi sıralıysa bir karşılaştırmada durur. Hocanın örneği `3, 5, 7, 9` gibi bir diziydi: 5 ile 3 karşılaştırılır, 5 büyüktür, başa kadar gitmeye gerek yoktur. 7 ile 5 karşılaştırılır; 7, 5'ten büyükse 3'ten de büyüktür. 9 ile 7 karşılaştırılır; 9, 7'den büyükse 5 ve 3'ten de büyüktür. Her adımda tek karşılaştırma yeter, n elemanlı dizide toplam yaklaşık n karşılaştırma yapılır. Best case $O(n)$'dir.
- **Bubble sort:** Ardışık elemanları ikişer ikişer karşılaştırarak ilerler. Hocaya göre dizinin sıralı olması bir şey değiştirmez, $O(n^2)$'dir (bkz. ders sonrası not).
- **Quicksort ve merge sort:** Bu yıl görülmedi, gelecek yılki algoritma analizi dersinde görülecek. Böl ve yönet (divide and conquer) yaklaşımıyla ilerledikleri için best case'leri $O(n \log n)$'dir.
- **Heap sort:** n kez kök ile bakılan bölgenin son elemanı yer değiştirilir ve heapify yapılır. Heapify $O(\log n)$ olduğu için toplam $O(n \log n)$'dir. Dizinin sıralı olup olmaması kontrol edilmez, bir şey ifade etmez.

Hoca shell sort'un görülmediğini söyleyip onu geçti.

2.3 Worst Case: Dizi Ters Sıralıysa

- **Selection sort:** Değişmez, $O(n^2)$.
- **Insertion sort:** Değişir. Dizi ters sıralıysa her eleman kendinden önce gelen bütün elemanlarla karşılaştırılır. Karmaşıklık $O(n^2)$'ye çıkar.
- **Bubble sort:** Değişmez, $O(n^2)$.
- **Quicksort:** Dizi ters yönde sıralı olursa karmaşıklık $O(n^2)$'ye çıkar (gelecek yıl görülecek).
- **Merge sort ve heap sort:** Değişmez, $O(n \log n)$.

2.4 Ek Bellek

Merge sort dışındaki algoritmalar, yer değiştirme sırasında yalnızca bir yedek değişken kullanır. Merge sort ise birleştirme (merge) işlemi için n elemanlı geçici bir dizi kullanır, dolayısıyla ek bellek ihtiyacı vardır.

Algoritma	Best case	Worst case	Ek bellek
Selection sort	$O(n^2)$	$O(n^2)$	Bir yedek değişken
Insertion sort	$O(n)$	$O(n^2)$	Bir yedek değişken
Bubble sort	$O(n^2)$ (hocaya göre)	$O(n^2)$	Bir yedek değişken
Quicksort	$O(n \log n)$	$O(n^2)$	Bir yedek değişken (hocaya göre)
Merge sort	$O(n \log n)$	$O(n \log n)$	n elemanlı geçici dizi
Heap sort	$O(n \log n)$	$O(n \log n)$	Bir yedek değişken

2.5 Hangisi Tercih Edilir?

$O(n \log n)$ karmaşıklıklı üç yöntem vardır: quicksort, merge sort ve heap sort. Heap sort bunlar arasında iyi bir konumdadır. Quicksort worst case'de $O(n^2)$'ye çıkarken heap sort her zaman $O(n \log n)$'dir. Merge sort'a göre de ek bellek açısından avantajlıdır, çünkü yalnızca bir yedek değişken kullanır.

Ama en sık kullanılan quicksort'tur, çünkü pratikte daha hızlı çalışır. Worst case olmasın diye, dizinin büyükten küçüğe sıralı gelme ihtimaline karşı farklı yöntemler kullanılır; örneğin dizi önce karıştırılabilir (shuffle). Böylece quicksort'un hep average case'de kalması sağlanmaya çalışılır. Bunlar gelecek dönem algoritma analizi dersinde anlatılacak. Heap sort ise ihtiyaca göre kullanılabilecek etkili bir algoritmadır.

> **Ders sonrası not:** Hoca bubble sort'u her durumda $O(n^2)$ olarak verdi. Bu, her geçişi sonuna kadar yapan sürüm için doğrudur. Bir geçişte hiç yer değiştirme olmazsa duran (bayraklı) sürümde sıralı dizi tek geçişte biter ve best case $O(n)$ olur. Python ile sayıldı: sıralı 10 elemanlı dizide 9, 100 elemanlı dizide 99 karşılaştırma yapılır.
>
> **Ders sonrası not:** Quicksort'ta pivot olarak ilk (ya da son) eleman seçilirse yalnızca ters sıralı dizi değil, zaten sıralı dizi de worst case'tir. Python ile sayıldı: ilk eleman pivot alındığında 100 elemanlı sıralı dizide de ters sıralı dizide de $100 \cdot 99/2 = 4950$ karşılaştırma yapılır.

3. Öncelikli Kuyruk ve Gerçekleştirme Seçenekleri

3.1 Neden Öncelikli Kuyruk?

Normal kuyrukta ilk gelen ilk çıkar. Hocanın örneği bankadaki müşteri kuyruğuydu: ilk gelen kişinin işi 3 dakika, ikincininki 5 dakika, üçüncününkü 17 dakika, dördüncününkü 4 dakika sürecek olsun. İşi biten kuyruktan çıkar, yeni gelen kuyruğun sonuna yerleşir.

Bazı durumlarda bu alışılmış sıraya uyulmaması gerekir:

- **Acil servis:** Ambulansla durumu çok acil bir hasta geldiğinde onu diğerlerinin önüne geçirirsiniz, çünkü önceliği vardır. Ama iş bununla bitmez. Acil serviste herkesin durumu acildir; o hastadan sonra kime bakılacağı da yine aciliyete göre seçilmelidir.
- **İşletim sistemleri:** Bazı durumlarda kısa olan iş öne geçirilebilir. Bazen de bir işe öncelik verilip kuyrukta bekleyen işlerden önce yapılması istenir.

Öncelikli kuyruk (priority queue), işlerin sırasının ihtiyaca göre önceliklerine göre belirlendiği yapıdır. Derste en acil olanın önceliği en yüksek (maksimum) kabul edildi. Soru, bunun hangi veri yapısıyla gerçekleştirileceğidir. Hoca dört seçenek için ekleme (insert), en öncelikliyi bulma ve en öncelikliyi çıkarma (remove) karmaşıklıklarını sınıfla birlikte değerlendirdi.

3.2 Sıralı Dizi

Elemanlar aciliyetlerine göre en yüksek öncelikli en önde olacak şekilde sıralı tutulur. Örnek dizi: `11, 9, 7, 2, 1`.

- **Ekleme:** Önceliği 8 olan biri geldiğinde doğru yeri bulmak için en kötü ihtimalle n kontrol yapılır. Karmaşıklık $O(n)$'dir. Bunu sınıftan bir öğrenci de "tek tek bakarsak $O(n)$" diye cevapladı.
- **En öncelikliyi bulma:** En başta olduğu için $O(1)$'dir.
- **En öncelikliyi çıkarma:** 11 çıkarılınca diğerlerini bir öne itmek gerekir (9, 7, 2 bir öne gelir). Bu durumda $O(n)$'dir. Ama dizi dairesel kuyruk (circular queue) olarak tutulursa yalnızca `front`'un gösterdiği yer değiştirilir, kaydırma yapılmaz ve çıkarma $O(1)$ olur.

Soru: Sıralı dizide ekleme, ikili arama (binary search) kullanılırsa $O(\log n)$'de yapılamaz mı?
Cevap: Ekleme yerini bulmak $O(\log n)$'dir; 8'in 9 ile 7 arasına girmesi gerektiği ikili aramayla bulunur. Ama 8'i yerleştirmek, ondan sonraki bütün elemanları bir kaydırmak demektir ve bunun çaresi yoktur. Dizi `11, 9, 8, 7, 2, 1` hâline gelmelidir. Toplam karmaşıklık $O(n+\log n)$, yani $O(n)$'dir. Hoca bunun güzel bir soru olduğunu, kendisinin bu noktayı hızlı geçtiğini ama farkına varılmasının önemli olduğunu söyledi.

Ders sonrası not: Hoca diziyi büyükten küçüğe sıralı tuttu ve çıkarmayı $O(1)$ yapmak için dairesel kuyruk önerdi. Dizi küçükten büyüğe sıralı tutulursa en öncelikli eleman dizinin sonunda durur; onu çıkarmak için yalnızca eleman sayısı bir azaltılır ve kaydırma gerekmeden $O(1)$ elde edilir. Ekleme yine $O(n)$ kalır.

3.3 Sıralı Bağlı Liste

Bağlı liste (linked list) yine önceliklerine göre büyükten küçüğe sıralı oluşturulur. Örnek liste: `17 -> 11 -> 9 -> 3 -> ...`.

- **Ekleme:** 8 geldiğinde doğru yeri bulmak için tek tek bakılır. En sona kadar gitmek gerekebilir. 8, 9 ile 3 arasına alınır: 9 ile 3 arasındaki bağ koparılır, 8 araya bağlanır. Karmaşıklık $O(n)$'dir.
- **En öncelikliyi çıkarma:** `head` 17'yi gösteriyor. Artık 11'i göstermesi sağlanır. $O(1)$.
- **En öncelikliyi bulma:** En büyük öncelikli eleman en baştadır. $O(1)$.

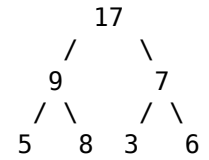
3.4 Sırasız Dizi

Elemanlar hiç sıralanmadan, geliş sırasıyla tutulur. Örnek dizi: `3, 7, 4, 6, 2`. Önceliği 8 olan biri gelirse sona girer.

- **Ekleme:** Gelen sona eklenir. $O(1)$.
- **En öncelikliyi bulma:** Bütün elemanlara bakmak gerekir. $O(n)$.
- **En öncelikliyi çıkarma:** Önce bulmak, sonra çıkarılan elemandan sonra gelenleri kaydırmak gerekir. Örneğin 8'i çıkarmak, 8'den sonra gelenleri kaydırmak demektir. Worst case'de $O(n)$.

3.5 İkili Heap

Hoca öncelikli kuyruğu bir max heap ile göstermeyi önerdi. Örnek heap: `17, 9, 7, 5, 8, 3, 6`. (Hoca önce 72 yazmaya başladı, sonra "çok büyük oldu" diyerek 17 yaptı.)



Ekleme, 13 geliyor: 13 önce en son göze, 5'in çocuğu olarak yerleştirilir. Sonra sift up yapılır:

- 13, ebeveyni 5'ten büyüktür: yer değiştirirler.
- 13, yeni ebeveyni 9'dan büyüktür: yer değiştirirler.
- 13, 17'den küçüktür: durulur.

Sonuç dizi (ara adım eklendi, python ile doğrulandı): `17, 13, 7, 9, 8, 3, 6, 5`. Karmaşıklık $O(\log n)$'dir.

En öncelikliyi çıkarma, 17 siliniyor: Hoca bu kez 13'ü eklenmemiş kabul edip ilk heap üzerinden gösterdi. Son eleman 6 köke alınır, 17 silinir. Sonra 6 sift down ile aşağı taşınır:

- 6'nın çocukları 9 ve 7'dir; büyük olan 9'dur. 9 ile 6 yer değiştirir.
- 6'nın yeni çocukları 5 ve 8'dir; büyük olan 8'dir. 8 ile 6 yer değiştirir.
- 6'nın altında çocuk kalmaz.

Sonuç: `9, 8, 7, 5, 6, 3` (python ile doğrulandı). Karmaşıklık $O(\log n)$'dir, çünkü sift down her seferinde yarıya bakarak iner. 9 ile 7 karşılaştırılıp 9'un büyük olduğu anlaşıldığı andan itibaren 7 tarafına hiç bakılmaz. Sonra 8 ile 5 karşılaştırılıp 8'in büyük olduğu anlaşıldığında 5'in tarafına (orada başka elemanlar olsaydı bile) hiç bakılmaz.

En öncelikliyi bulma: Kökte durduğu için $O(1)$.

3.6 Karşılaştırma ve Seçim

Yapı	Ekleme	En öncelikliyi bulma	En öncelikliyi çıkarma
Sıralı dizi	$O(n)$	$O(1)$	$O(n)$; dairesel kuyrukla $O(1)$
Sıralı bağlı liste	$O(n)$	$O(1)$	$O(1)$
Sırasız dizi	$O(1)$	$O(n)$	$O(n)$
İkili heap	$O(\log n)$	$O(1)$	$O(\log n)$

Öncelikli kuyruk için genellikle tercih edilen yapı ikili heap'tir (binary heap tree). Ama hangisinin kullanılacağı uygulamaya bağlıdır:

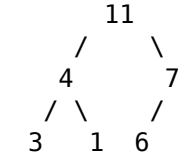
- Kuyruğa çok sık yeni eleman eklenmeyeceği biliniyorsa, yani bir kere sıralandıktan sonra çok nadiren yeni eleman geliyor ve genel olarak yapılacak iş sıradaki elemanı çıkarmaksa, basitçe sıralı dizi ya da sıralı liste kullanılabilir. Bu durumda ekleme ile pek uğraşılmaz.
- Çok hareket varsa, durmadan yeni elemanlar geliyorsa, örneğin bir acil servis ortamında, ikili heap daha etkilidir. Sürekli ekleme ve çıkarma yapıldığında en kârlı yapı odur.

4. Heap ile Öncelikli Kuyruk İşlemleri

Aradan sonra hoca, öncelikli kuyruk için en etkili veri yapılarından olan heap ağacının nasıl kullanılacağını adım adım gösterdi. Yine max heap kullanılır ve en öncelikli işin en yüksek değere sahip olduğu düşünülür. Heap üzerinde öncelikli kuyruk için iki işlem yapılır.

4.1 En Öncelikli İşi Kuyruktan Çıkarma

Örnek heap: `11, 4, 7, 3, 1, 6`.



En öncelikli iş kökteki 11'dir. Bunu çıkarmanın alışılmış yolu, son elemanı köke taşımaktır; en kolay taşıma budur. Sonra heapify (sift down) yapılır:

- 6 köke gelir. Çocukları 4 ve 7 karşılaştırılır; 7 büyüktür.
- 7 ile 6 yer değiştirir. 11 artık silinmiştir, onun yeri yoktur.
- Ağacın yeni hâli: `7, 4, 6, 3, 1` (python ile doğrulandı).

Hoca ardından işlemin adımlarını yazdı:

1. Kökteki değer döndürülür. Hizmet edilecek iş odur; örneğin hastanın adı, soyadı gibi bilgileri neyse onlar geri döndürülür.
2. Heap'in en son elemanı köke yazılır. Burada yer değiştirme yapmaya gerek yoktur: 11'i aşağı taşımaya gerek yoktur, son elemanı doğrudan köke yazmak yeter. Eleman sayısı n'den $n-1$'e düşer, 11 artık bizim için yoktur.
3. Sift down ile bu eleman, çocuklarından büyük olana kadar aşağı taşınır.

Hoca "çocuklarından büyük olana kadar" ifadesini bir ek durumla açıkladı. 6'nın sağ alt ağacında, 7'nin altında bir de 5 olsaydı, yani heap `11, 4, 7, 3, 1, 5, 6` olsaydı: 6 köke gelir, 4 ile 7'den büyük olan 7 bulunur, 7 ile 6 yer değiştirir. Sonra 6 ile 5 karşılaştırılır. 6, 5'ten büyük olduğu için artık aşağı taşınmaz. Yeni hâl `7, 4, 6, 3, 1, 5` olur (python ile doğrulandı).

4.2 Kuyruğa Yeni Eleman Ekleme

Öncelikli kuyruk bir heap ile gösterildiğine göre, yeni gelen eleman heap'e eklenir. Hocanın örneğinde kökü 22 olan bir heap'e önceliği 18 olan bir eleman geldi:

- 18 önce en son göze yerleştirilir.
- Sift up ile yukarı taşınır: 18, ebeveyni 17'den büyüktür, yer değiştirirler.
- 18 ile 22 karşılaştırılır. 18 küçük olduğu için bir şey değişmez, işlem biter.

Artık n yerine $n+1$ eleman vardır.

Ekleme adımları:

1. Gelen eleman heap'in en sonuna yerleştirilir, n bir artar.
2. Ağaç yukarı doğru heapify edilir (sift up).

4.3 Uygulama: Acil Servis

Bu yapı acil servise çok uygundur. Ambulansla getirilen hastaların bilgileri durumlarının aciliyetine göre bir heap ağacında saklanırsa, yeni gelen kişi de önceliğine göre yerleşir ve hizmet de önceliğe göre verilir.

5. Huffman Kodlama

Hoca öncelikli kuyruğun "çok daha değişik" bir uygulaması olarak Huffman kodlamayı (Huffman coding) anlattı. Öğrenciler bu konuyu ayrık matematik dersinde görmüştü. Bu derste amaç, Huffman ağacının öncelikli kuyrukla nasıl kurulduğunu görmekti.

Huffman kodlama 1950'li yıllarda MIT'de David Huffman tarafından geliştirilmiş bir sıkıştırma yöntemidir.

5.1 Kayıplı ve Kayıpsız Sıkıştırma

Veri sıkıştırmada amaç, bilgi için daha az yer tutulmasını sağlamaktır. İki tür sıkıştırma vardır:

- **Kayıpsız sıkıştırma (lossless compression):** Hiçbir bilgi kaybolmaz. Bütün bilgiler yeniden erişilebilir olmalıdır.
- **Kayıplı sıkıştırma (lossy compression):** Biraz bilgi kaybı olur, ama bu kayıp fark edilmeyecek, hissedilmeyecek düzeydedir. Hocanın örneği JPEG'di: bir görüntü JPEG ile sıkıştırıldığında, insan gözünün algılayamayacağı şekilde sıkıştırılır ve biz bunu fark etmeyiz.

Huffman kodlama kayıpsızdır. Bir metin Huffman kodlamayla kodlanır (encode) ve sıkıştırılmış metin (compressed text) elde edilir. Sonra çözülür (decode) ve sıkıştırması açılmış metin (uncompressed text) elde edilir. Açılan metin orijinal metne birebir eşitse sıkıştırma kayıpsızdır.

5.2 Fikir: Sık Karakterlere Kısa Kod

Her karakter normalde 8 bitle saklanır. Soru, bunun daha az bitle yapılıp yapılamayacağıdır. Fikir şudur: bütün karakterleri 8 bitle göstermek yerine sık kullanılanları daha az bitle, az kullanılanları daha çok bitle göstermek.

Hocanın örneği: bir yazıda A karakteri 100 kez, Q karakteri 3 kez geçsin.

- Hepsi 8 bitle saklanırsa: $103 \cdot 8 = 824$ bit.
- A 1 bitle, Q 8 bitle saklanırsa: $100 \cdot 1 + 3 \cdot 8 = 100 + 24 = 124$ bit.

Birdenbire çok ciddi bir yer kazanılır (python ile doğrulandı).

5.3 Sorun: Karakter Nerede Bitiyor?

Kodlar değişken uzunlukta olunca yeni bir sorun çıkar. Karakterler arka arkaya durur. A 1 bit, B 2 bit, C 3 bit tutuyorsa, bir bit dizisinde hangi harfin nerede başlayıp nerede bittiğini nasıl anlarız? Bir karakterin bittiğini ve diğerine geçildiğini anlamamızı sağlayacak akıllıca bir tasarım gerekir. Huffman bunun için prefix kod (prefix code) kullanır.

5.4 Prefix Kod

Prefix kodda hiçbir karakterin kodu, başka bir karakterin kodunun ön eki (başlangıcı) değildir.

- **Prefix olmayan örnek:** A = `0`, B = `1` ise C = `11` olamaz. `11` görüldüğünde iki tane B mi, bir tane C mi kastedildiği ayırt edilemez.

- **Prefix kod örneği:** A = `0`, B = `11`, C = `101`. `110101101` dizisi şöyle çözülür: `11` B, `0` A, `101` C, `101` C. Sonuç `BACC`'dir; hiçbir harf diğerinin başında olmadığı için karışma olmaz (python ile doğrulandı).
  - `11`, C'nin başında yoktur. C = `110` olsaydı bu kod kullanılamazdı.
  - `0` hiçbir kodun başında yoktur. B = `01` olsaydı kullanılamazdı, ama B = `11`'dir.

5.5 Örnek 1: "ADA AD CADDD"

Hoca karışıklık olmasın diye metinde B harfini kullanmadı; alfabe A, C, D ve boşluktan oluşuyor. Metin `ADA AD CADDD`'dir.

Adım 1: Frekansları bul. Her karakterin kaç kez geçtiği sayılır:

Karakter	Frekans
C	1
boşluk	2
A	4
D	5

Adım 2: Düğümleri öncelikli kuyruğa yerleştir. Her karakter için bir düğüm oluşturulur ve frekansına göre küçükten büyüğe sıralı öncelikli kuyruğa konur: `C1, boşluk2, A4, D5`.

Adım 3: Birleştir. Kuyrukta birden fazla düğüm olduğu sürece en küçük frekanslı iki düğüm çıkarılır, birleştirilir ve toplam frekansla kuyruğa doğru yere geri eklenir:

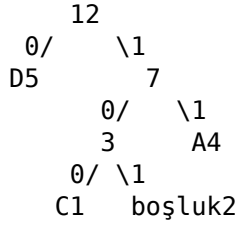
Adım	Birleştirilen	Yeni düğüm	Kuyruk
0			C1, boşluk2, A4, D5
1	C1 + boşluk2	3	3, A4, D5
2	3 + A4	7	D5, 7
3	D5 + 7	12	12

İlk adımda 1 ile 2'nin toplamı 3'tür. 3, kuyrukta 4 ve 5'ten küçük olduğu için yine en başta kalır. Sonra 3 ile 4 birleşir, 7 olur; 7, 5'ten büyük olduğu için 5'in arkasına yerleşir. Son olarak 5 ile 7 birleşir ve 12 olur. Başka düğüm kalmadığı için Huffman ağacı oluşmuştur.

Birleştirmede küçük olan sola, büyük olan sağa yazılır. Hoca bunun her zaman böyle olduğunu ve ayrık matematik dersinde de anlatıldığını söyledi.

Soru: Kuyrukta D neden öne geçti? **Cevap:** Çünkü 5 küçüktür. 3 ile 4 birleşip 7 olunca kuyruk `D5, 7` olur; 5, 7'den küçük olduğu için D önde durur. Hoca soruyu soran öğrencinin haklı olduğunu, kendisinin yazarken sıraları farklı yazdığını söyledi ve tahtayı düzeltti.

Adım 4: Kodları oku. Sola giden kenar 0, sağa giden kenar 1 alır. Semboller yapraklardadır.



Karakter	Frekans	Kod
D	5	`0`
A	4	`11`
C	1	`100`
boşluk	2	`101`

En çok kullanılan D bir bitle, ikinci sıradaki A iki bitle, daha az kullanılan C ve boşluk üçer bitle gösterilmiş olur.

Adım 5: Metni kodla.

```
A   D  A   boşluk A   D  boşluk C   A   D  D  D
11  0  11  101    11  0  101    100 11  0  0  0
```

Bit dizisi `11011101110101100011000` olur. Toplam 22 bittir; her karakter 8 bitle saklansaydı $12 \cdot 8 = 96$ bit gerekirdi (ara adım eklendi, python ile doğrulandı).

> **Ders sonrası not:** Hoca D'nin frekansını sayarken bir yerde "C'de 5 defa geçiyor" dedi; doğrusu D = 5, C = 1. Ayrıca tahtada boşluk için B, D için de birkaç yerde B yazıldı ve sonra düzeltildi. C'nin kodu bir an yanlış okundu, hoca `100` olarak düzeltti. Doğru tablo yukarıdakidir.

5.6 Kod Çözme

Kod çözülürken ağaçta kökten başlanır. 0 gelince sola, 1 gelince sağa gidilir. Bir yaprağa ulaşılınca o karakter yazılır ve yeniden köke dönülür.

Hocanın izlediği çözme:

- `1`, `1`: sağ, sağ, yaprak A.
- `0`: sol, yaprak D.
- `1`, `1`: A.
- `1`, `0`, `1`: boşluk.
- `1`, `1`: A. `0`: D.
- `1`, `0`, `1`: boşluk.
- `1`, `0`, `0`: C.
- `1`, `1`: A.
- `0`: D, `0`: D, `0`: D.

Sonuç yeniden `ADA AD CADDD`'dir (python ile doğrulandı).

> **Ek bilgi (derste anlatılmadı):** Huffman ağacında semboller yalnızca yapraklardadır. Bir sembolün kodu kökten o yaprağa giden yoldur. Bir yaprak başka bir yaprağın yolunun üzerinde bulunamayacağı için hiçbir kod başka bir kodun ön eki olamaz. Bu yüzden ağaçtan okunan kodlar her zaman prefix koddur ve çözme sırasında yaprağa ulaşıldığı anda karakter kesin olarak belirlenir.

5.7 Örnek 2: A = 3, B = 3, C = 1, D = 1, E = 2

Frekanslar küçükten büyüğe bir bağlı listede saklanır; bu bağlı liste öncelikli kuyruk olarak kullanılır. Hoca bağlı listenin iki yönlü de gösterilebileceğini, nasıl gösterildiğinin fark etmediğini söyledi.

Başlangıç kuyruğu: `C1, D1, E2, A3, B3`.

Adım	Birleştirilen	Yeni düğüm	Kuyruk
0			C1, D1, E2, A3, B3
1	C1 + D1	CD2	CD2, E2, A3, B3
2	CD2 + E2	CDE4	A3, B3, CDE4
3	A3 + B3	AB6	CDE4, AB6
4	CDE4 + AB6	CDEAB10	CDEAB10

En küçük iki düğüm C1 ve D1, bağlı listeden çıkarılır, birleştirilir ve CD2 olarak yeniden başa eklenir. Sonra CD2 ile E2 birleşir; 4 ettiği için A ve B'den sonra gelir. Sonra A ile B birleşir; 6 ettiği için CDE4'ten sonra gelir. Son olarak ikisi birleşir ve 10 olur.

```
        10
      0/  \1
    CDE4   AB6
```

```
     0/  \1   0/  \1
    CD2  E2  A3   B3
   0/ \1
  C1   D1
```

Karakter	Frekans	Kod
A	3	`10`
B	3	`11`
C	1	`000`
D	1	`001`
E	2	`01`

> **Ders sonrası not:** Hoca listeyi yazarken bir an "E3, F3" yazdı; verilen frekanslar A = 3, B = 3, C = 1, D = 1, E = 2'dir. Kodları okurken de C için "0000" dedi; ağaca göre C'nin kodu `000`, D'ninki `001`'dir. Python ile denetlendi: tablodaki kodlar prefix koddur ve toplam uzunluk $3 \cdot 2 + 3 \cdot 2 + 1 \cdot 3 + 1 \cdot 3 + 2 \cdot 2 = 22$ bit, Huffman'ın bu frekanslar için verdiği en küçük değere eşittir.

Hoca bu yapıda farklı bir şeye dikkat çekti. Öncelikli kuyruğu bu kez bağlı listelerle oluşturduk ve sonunda bir ağaç elde ettik. Bu ağaçta ebeveynlerin değerleri çocuklarınınkinden büyüktür: 10; 4 ve 6; 2, 2, 3, 3; 1, 1.

5.8 Öncelikli Kuyruğun Başka Kullanımları

Öncelikli kuyruk önümüzdeki haftalarda görülecek minimum yayılan ağaç (minimum spanning tree) algoritmalarında da kullanılacak. Hoca bunların da ayrık matematik dersinde görülmüş olabileceğini söyledi.

6. Sınava Hazırlık: Heap ve BST Soruları

Dersin son bölümünde hoca, sınav öncesi hazırlık olarak heap ağaçlarıyla ilgili birkaç soru çözdü. Bunların bazıları önceki yılların sınavlarında sorulmuş sorulardır.

6.1 Max Heap'te İkinci En Büyük Eleman

Hoca max heap'te ikinci en büyük elemanı bulmanın karmaşıklığını sordu. Bir öğrenci $O(1)$ dedi ve hoca cevabı doğruladı.

Örnek heap `25, 13, 8, 5, 7, ...` gibi altı elemanlıydı.

İkinci en büyük eleman kökün çocuklarından biridir. Ama önce durumlar düşünülmelidir:

- **Kökün iki çocuğu var ($n > 2$):** `ht[1]` ile `ht[2]` karşılaştırılır, büyük olan ikinci en büyüktür. Örnekte 13 > 8 olduğu için ikinci en büyük 13'tür. $n = 3$ olsaydı da iki çocuk olurdu.
- **Kökün tek çocuğu var ($n = 2$):** Başka şans yoktur, ikinci en büyük `ht[1]`'dir.
- **Kökün çocuğu yok ($n = 1$):** İkinci en büyük eleman yoktur.

Soru: İkinci senaryoda koşul $n = 2$ olmalı değil mi? **Cevap:** Evet, doğru. Hoca dalgınlıkla yanlış yazmıştı ve düzeltti: tek çocuk durumu $n = 2$'dir.

Durumların hepsini kontrol eden kod (`gcc -Wall` ile derlendi; $n = 5$, $n = 2$ ve $n = 1$ için denendi):

```c
#include <stdio.h>

/* Max heap'te ikinci en büyük eleman; bulunursa 1 döner ve max2'ye yazar */
int ikinci_buyuk(const int ht[], int n, int *max2) {
    if (n <= 1) {
        return 0;                 /* ikinci en büyük eleman yok */
    } else if (n == 2) {
        *max2 = ht[1];            /* kökün tek çocuğu var */
```

```
    } else {                        /* n > 2: kökün iki çocuğu var */
        if (ht[1] > ht[2]) *max2 = ht[1];
        else               *max2 = ht[2];
    }
    return 1;
}

int main(void) {
    int a[] = {25, 13, 8, 5, 7};
    int b[] = {25, 13};
    int c[] = {25};
    int m;
    if (ikinci_buyuk(a, 5, &m)) printf("a: %d\n", m);
    if (ikinci_buyuk(b, 2, &m)) printf("b: %d\n", m);
    if (!ikinci_buyuk(c, 1, &m)) printf("c: ikinci en buyuk yok\n");
    return 0;
}
```

Çıktı: `a: 13`, `b: 13`, `c: ikinci en buyuk yok`.

Hoca, eski yıllarda bu soruyu sorduklarında öğrencilerin neredeyse hepsinin yalnızca şu kısmı yazdığını anlattı:

```
if (ht[1] > ht[2]) max2 = ht[1];
else               max2 = ht[2];
```

Bu yanlıştır, çünkü üç durum vardır: iki çocuk olabilir, tek çocuk olabilir ya da hiç çocuk olmayabilir. Örneğin $n = 2$ iken `ht[2]` heap'in dışında kalır. Yanlış çalışmayan bir program için üç durumun da kontrol edilmesi gerekir. Yine de karmaşıklık $O(1)$'dir, çünkü yalnızca birkaç `if` ile n'e bakılır.

> **Hocanın vurgusu:** Ağaçlar üzerinde ilerlerken bir düğümün iki çocuğu mu, tek çocuğu mu var, yoksa hiç çocuğu yok mu, mutlaka kontrol edilmelidir. Bu, ağaçlarla ilgili sık yapılan bir hatadır; sınıfın neredeyse üçte ikisi bu soruda bu kontrolü atlar.

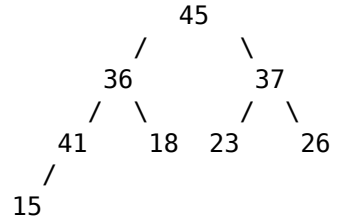
6.2 Bir Ağacın Heap Olup Olmadığını Kontrol Etme

Bu da eski sınav sorularından biridir. Dengeli (balanced) ve sola dayalı (left justified) olduğu bilinen bir ağaç verilir. Bu özellikler verildiği için yalnızca sayılara bakılır. Soru, ağacın (max) heap özelliğini taşıyıp taşımadığıdır.

Hoca önce birkaç yerinde hata olan bir ağaç çizip sordu.

Soru: Bu ağaç heap midir? **Cevap:** Bir öğrenci "değil, çünkü ne min heap ne max heap özelliğini sağlıyor" dedi. Hoca doğruladı. Sonra konuyu kolay izlemek için ağacı tek hatalı olacak şekilde değiştirdi.

Yeni ağaç dizide `45, 36, 37, 41, 18, 23, 26, 15` olarak saklanır ($n = 8$):



Öğrenciler hatanın 36'nın alt ağacında, 41'de olduğunu söyledi.

Bir öğrenci yukarıdan başlayarak her düğümü çocuklarıyla karşılaştırmayı önerdi. Hoca bunun doğru olduğunu, ama daha zor bir çözüm olduğunu söyledi: 45'ten başlayıp 36 ve 37'ye bakmak, sonra 36'dan 41 ve 18'e bakmak gerekir. Daha kolay yol, çocuklara bakmak yerine her elemanı ebeveyniyle karşılaştırmaktır. 36'dan başlanır: 36, 45'ten küçük mü? 37, 45'ten küçük mü? 41, 36'dan küçük mü? Hayır; hata bulundu.

> **Hocanın vurgusu:** Bir dizinin heap olup olmadığını kontrol ederken çocuklara bakmak yerine her elemanı ebeveyniyle karşılaştırmak daha kolaydır.

Algoritma:

- $i = 1$'den başlanır, `stop = 0` yapılır.
- $i \leq n - 1$ ve `stop` sıfır olduğu sürece devam edilir. Örnekte $n = 8$'dir, en son 15'e bakılır.
- i. elemanın ebeveyni $\lfloor (i-1)/2 \rfloor$. gözdedir. $i = 1$ için $(1-1)/2 = 0$, $i = 2$ için $(2-1)/2 = 0$, $i = 6$ için $(6-1)/2 = 2$.
- `ht[i]` ebeveyninden büyükse hata vardır, `stop = 1` yapılır ve döngüden çıkılır. Değilse i bir artırılır.
- Döngüden sonra `stop` 1 ise "i. eleman heap özelliğini bozuyor" yazılır, değilse "heap tree" yazılır.

Bu soruda heap özelliğini bozan tek bir eleman bulmak yeterlidir. Bütün hatalı elemanlar istenseydi döngü ilk hatada durdurulmazdı. Hoca `stop` değişkenini anlaşılır olsun diye kullandığını, kontrolü `while`'ın içinde yapmanın da sakıncası olmadığını söyledi.

```c
#include <stdio.h>

int main(void) {
    int ht[] = {45, 36, 37, 41, 18, 23, 26, 15};
    int n = sizeof(ht) / sizeof(ht[0]);
    int i = 1, stop = 0, parent;

    while (i <= n - 1 && !stop) {
        parent = (i - 1) / 2;
        if (ht[i] > ht[parent])
            stop = 1;              /* heap özelliği bozuk */
        else
            i++;
    }
    if (stop)
        printf("%d. eleman (%d) heap ozelligini bozuyor\n", i, ht[i]);
    else
        printf("Heap tree\n");
    return 0;
}
```

Kod `gcc -Wall` ile derlendi; çıktısı `3. eleman (41) heap ozelligini bozuyor`'dur.

Hoca algoritmayı sayılarla da izledi ve algoritma tasarlarken mutlaka bir kez sayılarla denemeyi tavsiye etti:

i	Ebeveyn $\lfloor (i-1)/2 \rfloor$	Karşılaştırma	Sonuç
1	0	36 > 45?	Hayır, i artar
2	0	37 > 45?	Hayır, i artar
3	1	41 > 36?	Evet, `stop = 1`

`stop` 1 olduğu için döngüden çıkılır ve $i = 3$ yazdırılır: üçüncü eleman ağacın heap özelliğini bozuyor.

> **Ders sonrası not:** Hoca izlemenin başında bir an "n'imiz yedi" dedi. Dizide 8 eleman vardır, $n = 8$'dir; son adres $n - 1 = 7$'dir. Hoca soruyu kurarken de "n'imiz 8, en son 15'e bakmamız lazım" demişti.

6.3 Bir Ağacın BST Olup Olmadığını Kontrol Etme

Hoca, önceki yıllarda aynı sorunun ikili arama ağacı (binary search tree, BST) için de sorulduğunu söyledi: verilen ağaç BST özelliğini bozuyor mu?

Ekrandaki ağacın kökü 21'di. Sol çocuğu 18, sağ çocuğu 32'ydi. 18'in çocukları 7 ve 19, 7'nin çocukları 3 ve 9'du. Hoca önce 7'nin altında hem 3 hem 5 olduğunu görünce "iki tane hata yapmayayım" diyerek 5'i 9 yaptı ve ağaca tek bir hata koydu.

Hocanın gösterdiği yöntem, her düğümün sol çocuğunun küçük, sağ çocuğunun büyük olup olmadığına bakmaktır: 21, 18 ve 32 ile; 18, 7 ve 19 ile; 32, 29 ve sağ çocuğu ile; 7, 3 ve 9 ile karşılaştırılır. İstenirse ters yönden, her eleman için adresine göre ebeveynine bakılarak da yapılabilir: sol çocuksa ebeveyninden küçük mü, sağ çocuksa büyük mü diye bakılır.

Soru: Bunun akış diyagramını yaparken yığın (stack) kullanmak gerekir mi? Hem sağ hem sol alt ağacı gezmek, sağa gittikten sonra sola dönmek gerekebilir. **Cevap:** Hayır. BST size dizi ya da bağlı liste olarak verilmiş olur ve bozulduğu yeri bulursunuz. İsterseniz özyinelemeyle (recursion) çözebilirsiniz. Ama özyineleme olmadan da çözülür: 21'in iki çocuğuna, 18'in iki çocuğuna, 32'nin iki çocuğuna, 7'nin iki çocuğuna sırayla bakarsınız. Aşağı inip geri dönmenize gerek yoktur. Ya da her eleman için yukarısına, yani ebeveynine bakarak gidersiniz.

Soru: 18'in altında 7 ve 19 var. 18'i 20 yapsak 7 kurala uyuyor ama 19 uymuyor. Hatalı düğüm 20 mi olur, 19 mu? **Cevap:** Fark etmez; "20 ile 19'u karşılaştırınca hata var" denebilir.

Soru: 19'u hatalı kabul edip artırmaya çalışırsak 20 ile yeni bir sorun çıkmaz mı? **Cevap:** Soru hata bulunduğunda biter. Düğüm öyle gelmiş, öyle yerleştirilmiş ve bir hata yapılmış. Onu değiştirmek ya da düzeltmek istenmiyor.

> **Ders sonrası not:** Her düğümü yalnızca doğrudan çocuklarıyla (ya da ebeveyniyle) karşılaştırmak BST kontrolü için yeterli değildir. BST kuralı bütün alt ağaç için geçerlidir: bir düğümün sol alt ağacındaki her değer ondan küçük, sağ alt ağacındaki her değer ondan büyük olmalıdır. Örneğin derste bozmak için kullanılan 25, 19'un sağ çocuğu olsaydı, her düğüm kendi çocuklarıyla uyumlu kalırdı; ama 25, 21'in sol alt ağacında olduğu hâlde 21'den büyüktür. Aşağıdaki kod `gcc -Wall` ile derlendi ve bu durumda yerel kontrolün 1 (BST), sınırlı kontrolün 0 (BST değil) döndürdüğü görüldü. Doğru yöntemler: inorder gezinme ile elde edilen dizinin artan olup olmadığına bakmak ya da özyinelemede her düğüme izin verilen bir alt ve üst sınır aktarmak.

```c
#include <stdio.h>
#include <limits.h>

typedef struct node {
    int key;
    struct node *left, *right;
} NODE;

/* Yerel kontrol: her düğüm yalnız doğrudan çocuklarıyla karşılaştırılır (yetersiz) */
int yerel_kontrol(NODE *p) {
    if (p == NULL) return 1;
    if (p->left != NULL && p->left->key > p->key) return 0;
    if (p->right != NULL && p->right->key < p->key) return 0;
    return yerel_kontrol(p->left) && yerel_kontrol(p->right);
}

/* Sınırlı kontrol: her düğüme izin verilen [alt, ust] aralığı aktarılır */
int bst_mi(NODE *p, long alt, long ust) {
    if (p == NULL) return 1;
    if (p->key < alt || p->key > ust) return 0;
    return bst_mi(p->left, alt, (long)p->key - 1) &&
           bst_mi(p->right, (long)p->key + 1, ust);
}

int main(void) {
    NODE n3 = {3, NULL, NULL}, n9 = {9, NULL, NULL};
    NODE n7 = {7, &n3, &n9}, n19 = {19, NULL, NULL};
    NODE n18 = {18, &n7, &n19}, n29 = {29, NULL, NULL}, n35 = {35, NULL, NULL};
    NODE n32 = {32, &n29, &n35}, kok = {21, &n18, &n32};
```

```
    NODE n25 = {25, NULL, NULL};

    printf("yerel=%d sinirli=%d\n", yerel_kontrol(&kok), bst_mi(&kok, LONG_MIN, LONG_MAX));
    n19.right = &n25;             /* 25, 21'in sol alt ağacına giriyor */
    printf("yerel=%d sinirli=%d\n", yerel_kontrol(&kok), bst_mi(&kok, LONG_MIN, LONG_MAX));
    return 0;
}
```

Çıktı: önce `yerel=1 sinirli=1`, 25 eklendikten sonra `yerel=1 sinirli=0`. (Kodda 32'nin sağ çocuğu için altyazıda geçen 35 kullanıldı.)

6.4 Ev Çalışması: Min Heap'te k. Küçük Eleman

Son olarak hoca çözmeyeceği bir soru verdi ve internetteki çözümlere bakmadan uğraşılmasını istedi: verilen bir min heap'te k. küçük elemanı bulun.

Hoca örnek olarak $k = 3$ için bir min heap çizdi ve birinci, ikinci, üçüncü küçük elemanları gösterdi. Sonra ağacı değiştirerek bu elemanların her zaman bu kadar düzgün yerlerde olmayacağını gösterdi: üçüncü küçük eleman ağacın farklı yerlerinde, farklı dallarda olabilir.

Çözüm, aşağıdaki Örnek Sorular bölümündedir.

Örnek Sorular

Bu derste ekran karesi ya da video kullanılamadı; aşağıdaki sorular derste sözlü sorulan ya da tahtada çözülen sorulardır.

Soru 1 (derste sözlü, 00:17:42): Heap'te heapify işleminin, örneğin kökteki bir elemanı aşağı taşımanın (sift down) karmaşıklığı nedir?

> **Çözüm:** $O(\log n)$. Sift down ve sift up her adımda bakılan bölgenin yarısını eler (Bölüm 1.4). Heap sort bu işlemi n kez yaptığı için her durumda $O(n \log n)$'dir.

Soru 2 (derste sözlü, 00:27:02): Öncelikli kuyruk büyükten küçüğe sıralı bir dizide tutuluyor (`11, 9, 7, 2, 1`) ve önceliği 8 olan biri geliyor. Ekleme ve en yüksek öncelikliyi bulmanın karmaşıklığı nedir?

> **Çözüm:** Ekleme $O(n)$'dir; doğru yer için en kötü ihtimalle n kontrol yapılır ve elemanlar kaydırılır. En yüksek öncelikli eleman baştadır, bulma $O(1)$'dir. Çıkarma kaydırmayla $O(n)$, dairesel kuyrukla $O(1)$'dir.

Soru 3 (derste sözlü, 00:31:12): Aynı sıralı dizide ekleme yeri ikili aramayla bulunursa ekleme $O(\log n)$ olur mu?

> **Çözüm:** Hayır. Yer $O(\log n)$'de bulunur, ama 8'in 9 ile 7 arasına girmesi için sonraki elemanlar kaydırılmalıdır. Toplam $O(n + \log n) = O(n)$'dir.

Soru 4 (derste sözlü, 00:34:03): Öncelikli kuyruk sırasız bir dizide (`3, 7, 4, 6, 2`) tutulursa ekleme, en öncelikliyi bulma ve çıkarma karmaşıklıkları nedir?

> **Çözüm:** Ekleme $O(1)$ (sona eklenir), bulma $O(n)$ (hepsine bakılır), çıkarma $O(n)$ (bulup sonra gelenleri kaydırmak gerekir).

Soru 5 (derste sözlü, 00:35:54): Max heap `17, 9, 7, 5, 8, 3, 6` ile tutulan öncelikli kuyruğa önceliği 13 olan eleman ekleniyor. Ekleme, en öncelikliyi (17) çıkarma ve en öncelikliyi bulma karmaşıklıkları nedir?

> **Çözüm:** 13 sona yazılır; 5 ve 9 ile yer değiştirir, 17'nin altında durur: `17, 13, 7, 9, 8, 3, 6, 5`. Ekleme $O(\log n)$. 17 çıkarılırken (ilk heap'ten) 6 köke gelir; 9 ile, sonra 8 ile yer değiştirir: `9, 8, 7, 5, 6, 3`. Çıkarma $O(\log n)$. Bulma kökte olduğu için $O(1)$. (Sonuç diziler python ile doğrulandı.)

Soru 6 (derste sözlü, 01:43:23): Max heap'te ikinci en büyük elemanı bulmanın karmaşıklığı nedir? Bulan kodu yazın.

Çözüm: $O(1)$. Üç durum kontrol edilmelidir: $n = 1$ ise ikinci en büyük yoktur; $n = 2$ ise `ht[1]`'dir; $n > 2$ ise `ht[1]` ile `ht[2]`'den büyük olandır. Kod Bölüm 6.1'dedir. Yalnızca son `if-else`'i yazmak yanlıştır.

Soru 7 (derste sözlü, 01:52:12; önceki yılların sınav sorusu): Dengeli ve sola dayalı olduğu bilinen, dizide `45, 36, 37, 41, 18, 23, 26, 15` olarak saklanan ağaç heap özelliğini taşıyor mu? Taşımıyorsa bozan ilk elemanı bulan algoritmayı yazın.

Çözüm: Taşımıyor. Her eleman ($i = 1, \ldots, n-1$) ebeveyni $\lfloor (i-1)/2 \rfloor$ ile karşılaştırılır. $i = 3$'te 41 > 36 bulunur: 3. eleman heap özelliğini bozuyor. Kod ve iz Bölüm 6.2'dedir.

Soru 8 (derste sözlü, 02:01:16; önceki yılların sınav sorusu): Verilen bir ikili ağaç BST özelliğini bozuyor mu? Bozan yeri bulun.

Çözüm: Derste her düğümün sol çocuğunun küçük, sağ çocuğunun büyük olup olmadığına bakıldı ve stack gerekmediği söylendi. Bu yerel kontrol yetersizdir; doğru çözüm her düğüme izin verilen alt-üst sınırı aktaran özyinelemeli kontrol ya da inorder dizinin artan olup olmadığına bakmaktır (Bölüm 6.3, ders sonrası not ve kod).

Soru 9 (derste sözlü, 02:08:34; ev çalışması): Verilen bir min heap'te k. küçük elemanı bulun (örnekte $k = 3$).

Çözüm (derste verilmedi): Derste görülen "en öncelikliyi çıkarma" işlemi $k-1$ kez uygulanır: her seferinde son eleman köke yazılır, eleman sayısı bir azaltılır ve sift down yapılır. Bunun sonunda kökte kalan eleman k. küçüktür. Asıl heap bozulmasın diye işlem bir kopya üzerinde yapılır. Kopyalama $O(n)$, her silme $O(\log n)$ olduğu için toplam $O(n + k \log n)$'dir. Aşağıdaki kod `gcc -Wall` ile derlendi ve 20.000 rastgele min heap üzerinde bütün k değerleri için sıralanmış diziyle karşılaştırıldı; hiç hata çıkmadı.

```c
#include <stdlib.h>

static void sift_down(int a[], int i, int n) {
    int kucuk, l, r, tmp;
    for (;;) {
        l = 2 * i + 1; r = 2 * i + 2; kucuk = i;
        if (l < n && a[l] < a[kucuk]) kucuk = l;
        if (r < n && a[r] < a[kucuk]) kucuk = r;
        if (kucuk == i) return;          /* çocuklarından küçük: dur */
        tmp = a[i]; a[i] = a[kucuk]; a[kucuk] = tmp;
        i = kucuk;
    }
}

/* Min heap'te k. küçük eleman (1 <= k <= n) */
int k_kucuk(const int ht[], int n, int k) {
    int *a = malloc(n * sizeof(int));
    int i, sonuc;
    for (i = 0; i < n; i++) a[i] = ht[i];   /* kopya */
    for (i = 1; i < k; i++) {               /* k-1 kez kökü sil */
        a[0] = a[n - 1];                    /* son eleman köke */
        n--;
        sift_down(a, 0, n);
    }
    sonuc = a[0];
    free(a);
    return sonuc;
}
```

Ek bilgi (derste anlatılmadı): k, n'e göre küçükse daha hızlı bir yol da vardır: yardımcı bir min öncelikli kuyruğa önce kök konur; k kez en küçük çıkarılır ve çıkarılan düğümün heap'teki çocukları yardımcı kuyruğa eklenir. k. çıkarılan eleman cevaptır.

Yardımcı kuyruk en fazla $k+1$ eleman tuttuğu için süre $O(k \log k)$ olur ve asıl heap kopyalanmaz.

Dersin Toparlaması

Ders, heap sort'un tekrarıyla başladı ve sıralama algoritmalarının best, worst case karmaşıklıkları ile ek bellek ihtiyaçları karşılaştırıldı. Heap sort her durumda $O(n \log n)$ olması ve yalnızca bir yedek değişken kullanmasıyla etkili bir yöntemdir; pratikte en sık kullanılan ise quicksort'tur. Ardından öncelikli kuyruk tanıtıldı. Sıralı dizi, sıralı bağlı liste, sırasız dizi ve ikili heap ile gerçekleştirmenin karmaşıklıkları karşılaştırıldı. Ekleme ve çıkarmanın sık olduğu durumlarda ikili heap en uygun yapıdır: ekleme son göze yazıp sift up, çıkarma son elemanı köke yazıp sift down ile $O(\log n)$'de yapılır.

Öncelikli kuyruğun farklı bir uygulaması olarak Huffman kodlama işlendi. Sık geçen karakterlere kısa, seyrek geçenlere uzun prefix kodlar verilir; en küçük frekanslı iki düğüm tekrar tekrar birleştirilerek Huffman ağacı kurulur ve kodlar ağaçtan okunur. Son bölümde sınava hazırlık için max heap'te ikinci en büyük eleman, heap kontrolü ve BST kontrolü soruları çözüldü; min heap'te k. küçük eleman ev çalışması olarak bırakıldı. Öncelikli kuyruklar önümüzdeki haftalarda minimum yayılan ağaç algoritmalarında yeniden kullanılacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.